

业务 Tab 接入协议 v1.0-draft

状态：草案，待团队 review 负责人：<张琦> 最后更新：2026-05-28

一、协议概览

本协议定义业务模块接入客户端容器的标准化方式。接入方只需按协议声明 Tab 元信息、实现生命周期回调、配置扩展点，即可将业务页面嵌入容器，无需修改容器核心代码。

核心参与者



- 容器**：负责 Tab 注册管理、页面切换、TitleBar 渲染、生命周期调度
- 接入方**：实现协议接口，提供页面 Composable、响应容器生命周期

二、Tab 元信息

2.1 TabDefinition

代码块

```
1  @Stable
2  data class TabDefinition(
3      val id: String,                // 唯一标识, 如 "com.example.content-audit"
4      val displayName: String,       // 标题栏显示名称
5      val icon: ImageVector,         // BottomBar 图标
6      val route: String,             // 路由路径, 如 "/content-audit"
7      val version: SemanticVersion,  // 协议版本
8      val minContainerVersion: Int,  // 最低容器版本
9      val permissions: List<String>, // 所需权限 (网络、存储等)
```

```

10     val extension: TabExtension?           // 扩展点配置
11 ) {
12     val fullId: String get() = "$$id@$$version"
13 }
14
15 @Stable
16 data class SemanticVersion(
17     val major: Int,
18     val minor: Int,
19     val patch: Int = 0
20 ) {
21     override fun toString() = "$$major.$$minor.$$patch"
22 }

```

2.2 字段约束

字段	必填	说明
id	是	反向域名格式，全局唯一
displayName	是	≤ 16 字符，容器 TitleBar 用
icon	是	24dp × 24dp
route	是	/ 开头，用于路由分发
version	是	协议版本号，容器据此做兼容处理
minContainerVersion	否	默认 1，低于此版本容器拒绝加载
permissions	否	如 ["android.permission.CAMERA"]
extension	否	null 则使用容器默认行为

三、生命周期

3.1 回调定义

代码块

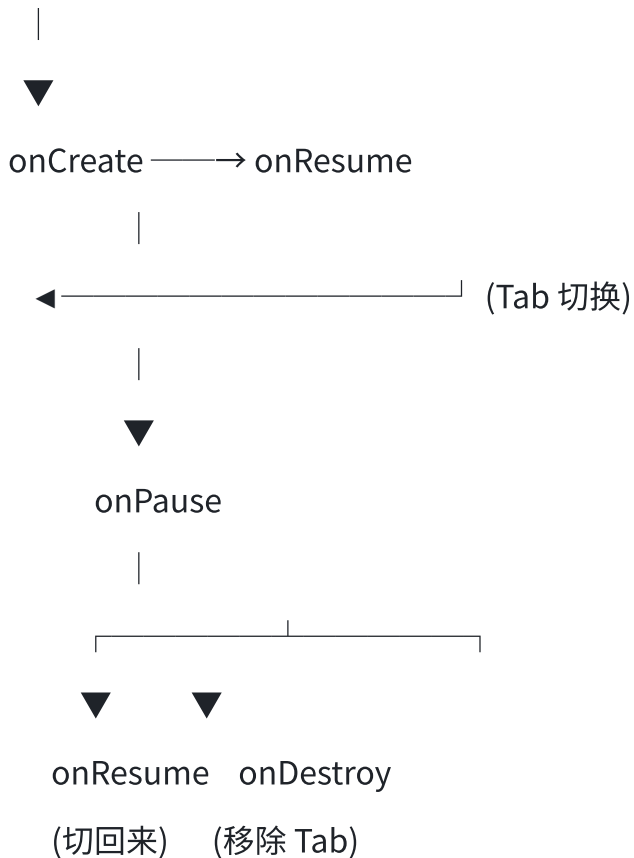
```

1  interface TabLifecycle {
2      fun onCreate(bundle: Bundle?)           // Tab 首次创建
3      fun onResume()                         // Tab 可见（切换回来）
4      fun onPause()                         // Tab 不可见（切换走）
5      fun onDestroy()                       // Tab 被销毁
6      fun onChange(config: Configuration) = {} // 配置变更（横竖屏等），可选

```

3.2 生命周期状态机

Tab 创建



3.3 调度规则

1. **首次进入**: `onCreate` → `onResume`
2. **Tab 间切换**: 当前 `Tab.onPause` → 新 `Tab.onResume` (若新 `Tab` 首次则先 `onCreate`)
3. **退出 Tab**: `onPause` → `onDestroy`
4. **超时保护**: 每个回调执行上限 5 秒, 超时容器自动降级处理 (显示兜底页面)
5. **异常保护**: 回调抛异常不影响其他 `Tab`

四、UI 扩展点

4.1 扩展点类型

```

1  data class TabExtension(
2      val titleBar: TitleBarExtension?,    // TitleBar 右侧按钮/菜单
3      val fab: FabExtension?,              // 悬浮按钮
4      val bottomPanel: BottomPanel?        // 底部扩展面板
5  )
6
7  data class TitleBarExtension(
8      val rightIcon: ImageVector?,         // 右侧单个图标按钮
9      val rightText: String?,              // 右侧文字按钮
10     val menuItems: List<MenuItem> = emptyList() // 右侧溢出菜单 (三点)
11 ) {
12     // rightIcon 和 rightText/menuItems 互斥: rightIcon 不为 null 时, 忽略
    menuItems
13 }
14
15 data class MenuItem(
16     val id: String,
17     val label: String,
18     val onClick: () -> Unit
19 )
20
21 data class FabExtension(
22     val icon: ImageVector,
23     val label: String,
24     val onClick: () -> Unit
25 )
26
27 data class BottomPanel(
28     val content: @Composable () -> Unit,
29     val defaultHeight: Int // 默认面板高度 dp
30 )

```

4.2 扩展点行为

扩展点	默认行为	自定义后
TitleBar 右侧	无按钮	显示接入方指定的图标/文字/菜单
TitleBar 标题	TabDefinition.displayName	通过回调实时更新
FAB	无	右下角悬浮按钮
BottomPanel	无	页面底部扩展面板

4.3 已预留、本期不支持

这些扩展点不保证实现，留给后续版本：

- BottomBar 自定义 Tab（本期只支持预定义的几个 MainPageTab）

- 自定义 Drawer 内容
- Tab 间通信 / 数据共享

五、配置格式

5.1 静态注册（代码方式）

代码块

```
1  // 接入方在自己的模块中实现
2  object ContentAuditTab {
3      val definition = TabDefinition(
4          id = "com.example.content-audit",
5          displayName = "内容审核",
6          icon = Icons.Filled.Description,
7          route = "/content-audit",
8          version = SemanticVersion(1, 0, 0),
9          minContainerVersion = 1,
10         permissions = listOf("android.permission.INTERNET"),
11         extension = TabExtension(
12             titleBar = TitleBarExtension(
13                 menuItems = listOf(
14                     MenuItem("filter", "筛选", { /* ... / } ),
15                     MenuItem("stats", "统计", { / ... */ })
16                 )
17             )
18         )
19     )
20
21     @Composable
22     fun Page() {
23         // 接入方的 Composable 页面
24     }
25
26     val lifecycle = object : TabLifecycle {
27         override fun onCreate(bundle: Bundle?) { /* 初始化 / }
28         override fun onResume() { / 显示时刷新数据 / }
29         override fun onPause() { / 保存状态 / }
30         override fun onDestroy() { / 清理资源 */ }
31     }
32 }
```

5.2 动态注册（JSON 配置）

代码块

```
1  {
2    "id": "com.example.dashboard",
3    "displayName": "数据看板",
4    "icon": "dashboard",
5    "route": "/dashboard",
6    "version": { "major": 1, "minor": 0 },
7    "minContainerVersion": 1,
8    "permissions": [],
9    "extension": {
10      "titleBar": {
11        "rightText": "刷新"
12      }
13    }
14  }
```

5.3 推荐方式

- **静态注册**：接入方的页面和逻辑用 Kotlin 代码实现（类型安全、IDE 检查）
- **动态注册**：作为静态注册的补充，用于服务端下发的 Tab 列表（下发哪些 Tab 可用、版本号、排序等）

六、路由

6.1 路由规则

容器内路由格式: `/[tab-route]?[params]`

示例: `/content-audit?filter=pending`

- Tab 注册时声明 `route` 前缀
- 容器根据 `route` 前缀匹配分发到对应 Tab
- 查询参数由 Tab 自行解析（容器透传）

6.2 导航方式

```
代码块/ 容器内 Tab 间切换
2  container.switchTab(route = "/content-audit")
3
4  // Tab 内部跳转独立 Activity（如详情页）
5  // 由接入方自行 startActivity，容器不拦截
```

七、兼容策略

7.1 版本号语义

major.minor.patch

major: 不兼容变更（容器必须升级才能加载）

minor: 新增可选字段/回调（老容器忽略即可）

patch: 文档修正、Bug 描述（无行为变化）

7.2 向后兼容规则

场景	容器行为
容器版本 < Tab.minContainerVersion	拒绝加载，Toast 提示 "请升级客户端"
Tab 发送了容器不认识的字段	容器忽略多余字段
容器新增字段，Tab 未提供	容器使用默认值
Tab 调用不存在的扩展点	容器静默忽略

7.3 废弃流程

v1.x 标记 @Deprecated → v2.0 正式移除

示例：

字段 oldName → 保留两个版本，v1.2 标记 deprecated，v2.0 删除

八、错误码

错误码	含义	容器行为
1001	Tab ID 重复注册	拒绝第二个，Toast 警告
1002	minContainerVersion 不满足	拒绝加载，提示升级
1003	Tab 缺失必填字段	拒绝加载
1004	生命周期回调超时 (>5s)	显示兜底页面
1005	生命周期回调抛出异常	显示兜底页面，打印日志
1006	Tab 权限不足	拒绝加载，提示用户授权

九、接入流程（三步概览）

第一步：实现 TabLifecycle 接口，提供 Composable 页面

第二步：声明 TabDefinition 元信息

第三步：调用 Container.registerTab(definition, lifecycle, page)

完整示例代码见 `docs/dev-guide.md`（待编写）。